

Proyecto 2 IPC2: Sistema de Administración de Catálogo de Librería

Introducción

En el contexto de la ciencia de la computación y la ingeniería de software, la gestión eficiente de datos estructurados constituye uno de los pilares fundamentales para el desarrollo de soluciones robustas y escalables. El presente ensayo técnico aborda el diseño, implementación y evaluación del software desarrollado para el Proyecto 2 del curso Introducción a la Programación y Computación 2 (IPC2). La aplicación tiene como propósito principal la administración del catálogo completo de una librería mediante la integración del lenguaje C# en plataforma .NET.

Un requisito crítico de esta implementación es la abstención estricta del uso de colecciones genéricas nativas o bibliotecas de estructuras de datos predefinidas en el framework (tales como `List<T>`, `LinkedList<T>`, o `Dictionary<TKey,TValue>`). En su lugar, la arquitectura del sistema reposa íntegramente sobre Tipos de Datos Abstractos (TDA) propios, construidos desde cero mediante gestión explícita de memoria referencial (nodos y punteros lógicos). Asimismo, el sistema implementa la persistencia de información y el intercambio de datos mediante archivos estructurados en formato XML, complementado con un módulo de visualización gráfica basado en Graphviz (lenguaje DOT) para la auditoría y generación de reportes visuales de las estructuras internas en tiempo de ejecución.

Objetivos

Objetivo General

Desarrollar una aplicación de consola/GUI orientada a objetos en C# para la administración integral de un catálogo de librería, utilizando únicamente Tipos de Datos Abstractos (TDA) lineales y no lineales propios, con soporte para persistencia de datos en formato XML y generación de diagramas dinámicos con Graphviz.

Objetivos Específicos

- Diseñar e implementar estructuras de datos dinámicas personalizadas (Listas Enlazadas Simples, Listas Doblemente Enlazadas, Listas Circulares y/o Matrices Dispersas/Ortogonales) sin depender de librerías nativas del lenguaje.
- Construir un parser e intérprete de archivos XML utilizando flujos de lectura/escritura (`System.Xml` elemental o manipulación mediante cadenas) para la carga masiva y exportación del catálogo.
- Integrar la herramienta Graphviz mediante la generación automática de scripts con sintaxis DOT y su posterior ejecución por consola para renderizar representaciones visuales exactas del estado de las estructuras.

- Analizar la complejidad algorítmica (Notación Big O) de las operaciones fundamentales de inserción, búsqueda, modificación y eliminación sobre los TDA implementados.

Desarrollo Técnico

1. Arquitectura del Sistema y Tipos de Datos Abstractos (TDA)

El sistema se diseñó bajo una arquitectura en capas, separando la lógica de presentación, la lógica de negocio y la capa de almacenamiento/estructuras de datos.

La base de la persistencia en memoria principal se estructura a partir de nodos genéricos o específicos que enlazan referencias de objetos Libro, Categoría, Usuario o Transaccion.

Lista Doblemente Enlazada Cabeza-Cola (ListaDoble)

Se eligió una Lista Doblemente Enlazada para almacenar la lista principal de libros debido a la necesidad de realizar recorridos bidireccionales y optimizar la eliminación de elementos cuando se conoce la referencia directa del nodo.

- **NodoDoble:** Contiene la carga útil (**Data**), la referencia al nodo siguiente (**Siguiente**) y al nodo anterior (**Anterior**).
- **Operaciones Clave:**
 - **InsertarAlFinal(T data):** Asigna la nueva referencia al puntero **Cola** existente y actualiza enlaces en $O(1)$.
 - **EliminarPorId(string id):** Recorre la lista hasta encontrar la coincidencia y reajusta los punteros vecinos en $O(n)$ peor caso, $O(1)$ caso ideal.

Lista Lista Enlazada Simple (ListaSimple)

Utilizada para manejar elementos con menor volumen de recorrido bidireccional, como categorías o colas de espera/procesos dentro de la aplicación.

- **NodoSimple:** Contiene la carga útil (**Data**) y la referencia al nodo siguiente (**Siguiente**).

2. Procesamiento de Archivos XML

El módulo de entrada/salida implementa la persistencia de datos. Mediante el uso de XmlDocument o XmlTextReader (restringido únicamente al parseo sintáctico sin utilizar almacenamiento coleccionable de .NET), el sistema lee la estructura jerárquica del archivo de entrada: <libreria>

```
<categorias>
```

```
<categoria id="1" nombre="Ficción">
```

```
<libro isbn="978-0141182605">
```

```

        <titulo>1984</titulo>

        <autor>George Orwell</autor>

        <precio>15.99</precio>

        <existencias>25</existencias>

    </libro>

</categoria>

</categorias>

</libreria>

```

El algoritmo de carga realiza un recorrido recursivo o iterativo sobre las etiquetas. Por cada nodo DOM detectado, instancia la entidad correspondiente (**Libro**, **Categoria**) y la inserta directamente en el TDA personalizado correspondiente.

3. Integración con Graphviz para Representación Visual

Para validar la integridad de las referencias internas de los TDA, se construyó la clase **GraphvizService**. Esta clase recorre cada nodo del TDA activo y genera una cadena de texto en formato DOT: digraph G {

```

    node [shape=record, style=filled, fillcolor=lightgray];

    rankdir=LR;

    node0 [label="{<ant> | ISBN: 978-0141182605 \n 1984 | <sig>}"];

|---|

    node0:sig → node1:ant;

    node1:ant → node0:sig;

}

```

Posteriormente, la aplicación invoca la herramienta de consola de Graphviz mediante la clase **System.Diagnostics.Process**: `ProcessStartInfo startInfo = new ProcessStartInfo();`

```
startInfo.FileName = "dot.exe";
```

```
startInfo.Arguments = "-Tpng estructura.dot -o estructura.png";
```

Process.Start(startInfo);

Análisis de Eficiencia

A continuación, se detalla la complejidad algorítmica en tiempo y espacio (Notación Big O) para las operaciones sobre las estructuras de datos propias desarrolladas:

Estructura / Operación	Inserción	Búsqueda	Eliminación	Acceso por índice	Complejidad espacial
Lista Simple (Inicio)	O(1)	O(n)	O(1)	O(n)	O(n)
Lista Simple (Final sin cola)	O(n)	O(n)	O(n)	O(n)	O(n)
Lista Doble (Inicio/Final con cabeza/cola)	O(1)	O(n)	O(1)*	O(n)	O(n)
Búsqueda por ISBN/ID	-	O(n)	-	-	O(1)

**Nota: Se asume que se posee la referencia directa al nodo a eliminar. En caso contrario, requiere una búsqueda previa O(n).*

Justificación de Elección de Estructuras

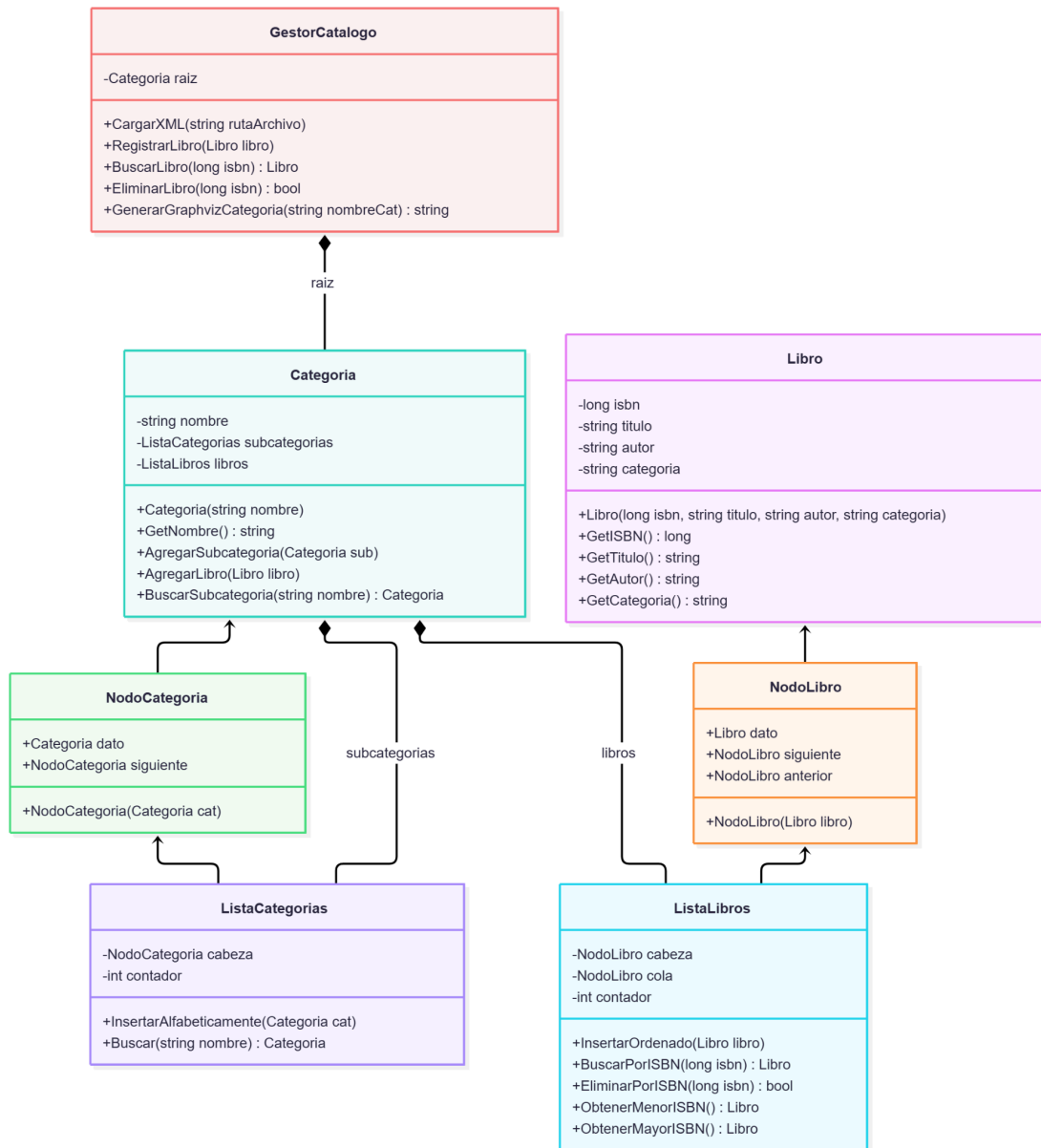
El uso de una **Lista Doblemente Enlazada** para la entidad central del catálogo (libros) se justifica por el requerimiento de permitir navegaciones hacia adelante y hacia atrás en la interfaz gráfica, logrando un costo constante O(1) al moverse entre nodos adyacentes (Siguiente / Anterior).

Conclusiones

- 1 La implementación de Tipos de Datos Abstractos (TDA) propios en C# permite un control total sobre la gestión de memoria y el puntero referencial, prescindiendo del nivel de abstracción de las colecciones genéricas de .NET y garantizando la comprensión profunda de las estructuras dinámicas.
- 2 El uso del formato XML como estándar de persistencia proporciona una solución portable e interoperable para la carga masiva y exportación de la información de la librería, permitiendo reconstruir el catálogo en memoria tras el procesamiento.
- 3 La automatización de diagramas mediante Graphviz valida la correcta reconexión de punteros, especialmente en operaciones de eliminación e inserción ordenada, y facilita la verificación de la integridad de las estructuras.

Anexos

- **Anexo A:** Diagrama de Clases UML del Sistema General.



- **Anexo B:** Manual de Usuario y Formato Especificado para el Archivo XML de Entrada.

